

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

LAB TASK 2

BECE406E

FPGA System Design Lab

NAME	KAARMUKILAN A G
REGISTRATION NO	23BEC0137
SLOT	L33+L34
FACULTY NAME	AARTHY M

AIM:

i) Design a Coke Vending machine using Mealy and Moore model which accepts Rs.5 and Rs.10 denominations only and deliver coke for Rs.15 and change if required.
ii) Verify the functionality of coke vending machine using Mealy and Moore model using Modelsim.
iii) Implement the Coke Vending machine using Mealy and Moore model in DE2-115 Board using Intel Quartus Prime.
iv) Attach the RTL view and Screenshots of code and outputs of Coke Vending machine.

SOFTWARE DETAILS:

Functional Simulation: Modelsim
Synthesis & Implementation: Intel Quartus Prime 23.1

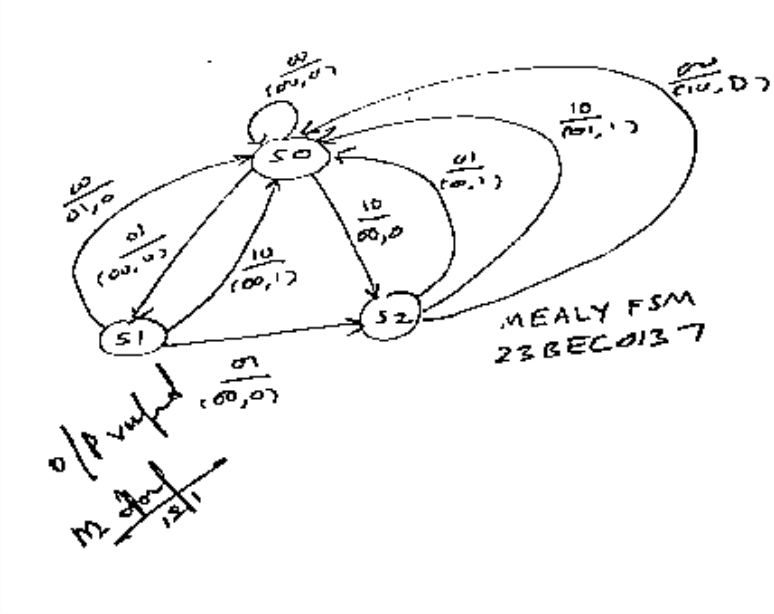
HARDWARE DETAILS:

Family: Cyclone IV E
Package: FBGA
Pin count: 780

Core speed grade – 7
Device - EP4CE115F29C7

VENDING MACHINE - MEALY

ATTESTATION - MEALY



RTL CODE and TESTBENCH

```
//vending machnie fsm -
/*
CHANGE 00 ,01,10= 0 RS,5RS,10RS;
23BEC0137 KAARMUKILAN A G
*/
module fsm(rst,clk,x,out,change);
input clk,rst;
input [1:0]x;
output reg out;
output reg [1:0] change;
reg[1:0] p_state,n_state;

parameter S0=2'b00;
parameter S1=2'b01;
parameter S2=2'b10;

always @(posedge clk or posedge rst)begin
 if(rst)
   p_state <= S0;
      else
   p_state <= n_state;
 end

  always @(p_state or x)
  begin
            n_state = S0;
         change= 2'b00;
case(p_state)
S0: if(x == 2'b01) begin //Received 5 Rs
 n_state = S1;
 change = 2'b00;
 out = 1'b0;
end
else if(x == 2'b10) begin   //Received 10 Rs
 n_state = S2;
 change = 2'b00;
 out = 1'b0;
end
else begin
 n_state = S0;
 change = 2'b00;
 out = 1'b0;
end
S1: if(x == 2'b01) begin //Received 10(5+ 5) Rs
 n_state = S2;
 change = 2'b00;
 out = 1'b0;
end
else if(x == 2'b10) begin             //Received 15(5+10) Rs
 n_state = S0;
 out = 1'b1;
 change = 2'b00;
end
else begin
 n_state = S0;
 change = 2'b01;
 out = 1'b0;
end
S2: if(x == 2'b01) begin //Received 15(5+ 5 + 5) Rs
 n_state = S0;
 out = 1'b1;
change=2'b00;
end
else if(x == 2'b10) begin //Received 20(10+ 10) Rs
 n_state = S0;
 out = 1'b1;
 change = 2'b01;
end
else begin
 n_state = S0;
 change = 2'b10;
 out = 1'b0;
end
endcase
end
endmodule
```

```
`timescale 1ns/1ps

module tb_fsm;
reg clk;
reg rst;
reg [1:0] x;
wire out;
wire [1:0] change;

fsm dut (rst,clk,x,out,change);

always #5 clk = ~clk;

    initial begin
        // Init
        clk = 0;
        rst = 1;
        x   = 2'b00;

        // Apply reset
        #15;
        rst = 0;
//Apply 5rs for 3 clks
        x = 2'b01;
        #30;   // 3 clock cycles

        // Apply 10 Rs (10) for 3 clocks

        x = 2'b10;
        #30;   // 3 clock cycles

        #20;
        $stop;
    end

endmodule
```

```
//vending machnie fsm -
/*
CHANGE 00 ,01,10= 0 RS,5RS,10RS;
23BEC0137 KAARMUKILAN A G
*/
module fsm(rst,clk,x,out,change);
input clk,rst;
input [1:0]x;
output reg out;
output reg [1:0] change;
reg[1:0] p_state,n_state;

parameter S0=2'b00;
parameter S1=2'b01;
parameter S2=2'b10;

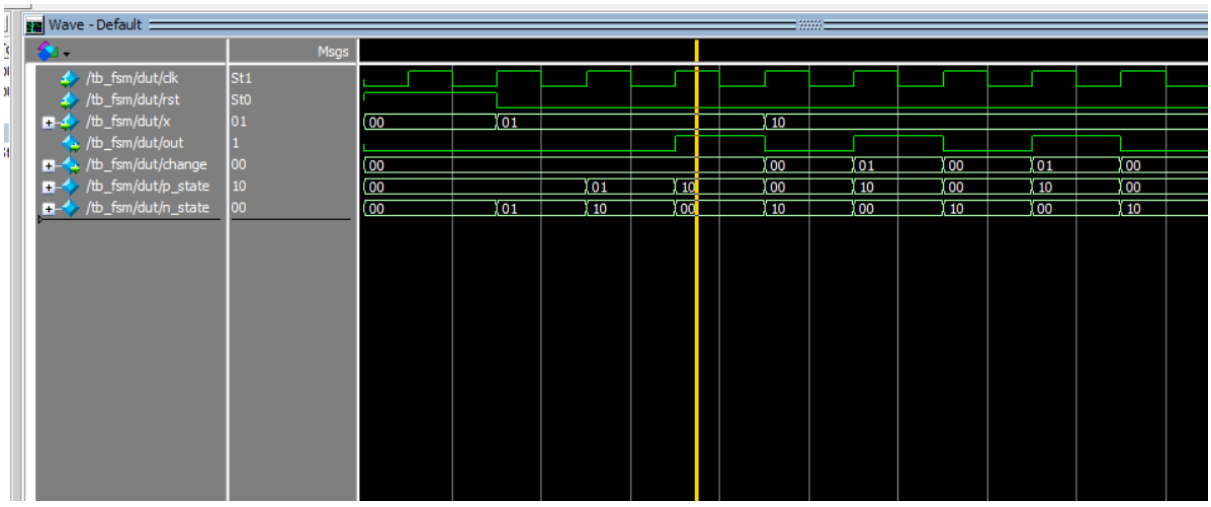
always @(posedge clk or posedge rst)begin
 if(rst)
   p_state <= S0;
     else
   p_state <= n_state;
 end

  always @(p_state or x)
  begin
           n_state = S0;
        change= 2'b00;
case(p_state)
S0: if(x == 2'b01) begin //Received 5 Rs
 n_state = S1;
 change = 2'b00;
 out = 1'b0;
end
else if(x == 2'b10) begin   //Received 10 Rs
 n_state = S2;
 change = 2'b00;
 out = 1'b0;
end
else begin
 n_state = S0;
 change = 2'b00;
 out = 1'b0;
end
S1: if(x == 2'b01) begin //Received 10(5+ 5) Rs
 n_state = S2;
 change = 2'b00;
 out = 1'b0;
end
else if(x == 2'b10) begin           //Received 15(5+10) Rs
 n_state = S0;
 out = 1'b1;
 change = 2'b00;
end
else begin
 n_state = S0;
 change = 2'b01;
 out = 1'b0;
end
S2: if(x == 2'b01) begin //Received 15(5+ 5 + 5) Rs
 n_state = S0;
 out = 1'b1;
change=2'b00;
end
else if(x == 2'b10) begin //Received 20(10+ 10) Rs
 n_state = S0;
 out = 1'b1;
 change = 2'b01;
end
else begin
 n_state = S0;
 change = 2'b10;
 out = 1'b0;
end
endcase
end
endmodule
```

```
`timescale 1ns/1ps
module tb_fsm;
    reg clk;
    reg rst;
    reg [1:0] x;
    wire out;
    wire [1:0] change;
    // DUT instantiation
    fsm dut (rst,clk, x,out, change );
   always #5 clk = ~clk;
    initial begin
        // Init
        clk = 0;
        rst = 1;
        x   = 2'b00;
                #15;
        rst = 0;
        // Apply 5 Rs 03 clocks
        x = 2'b01;
        #30;   // 3 clock cycles (3 × 10 ns)
        // Apply 10 Rs) for 3 clocks
        x = 2'b10;
        #30;   // 3 clock cycles
        // Stop simulation
        #20;
        $stop;
    end

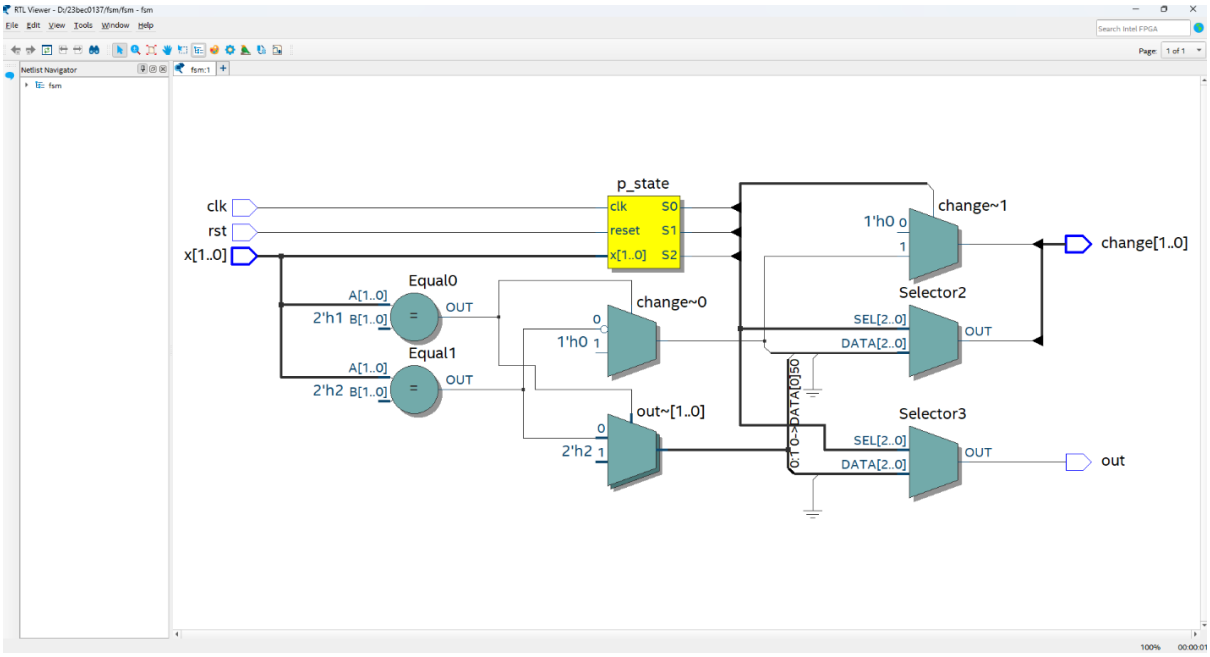
endmodule
```

FUNCTIONAL SIMULATION:

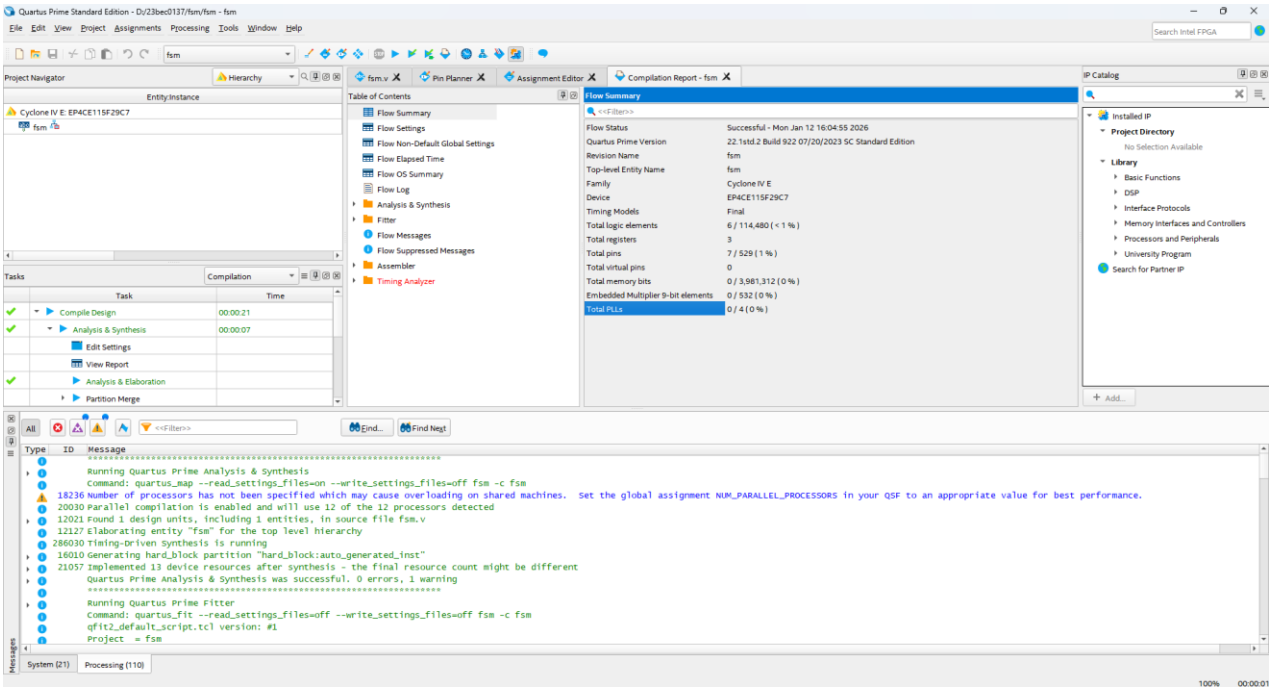


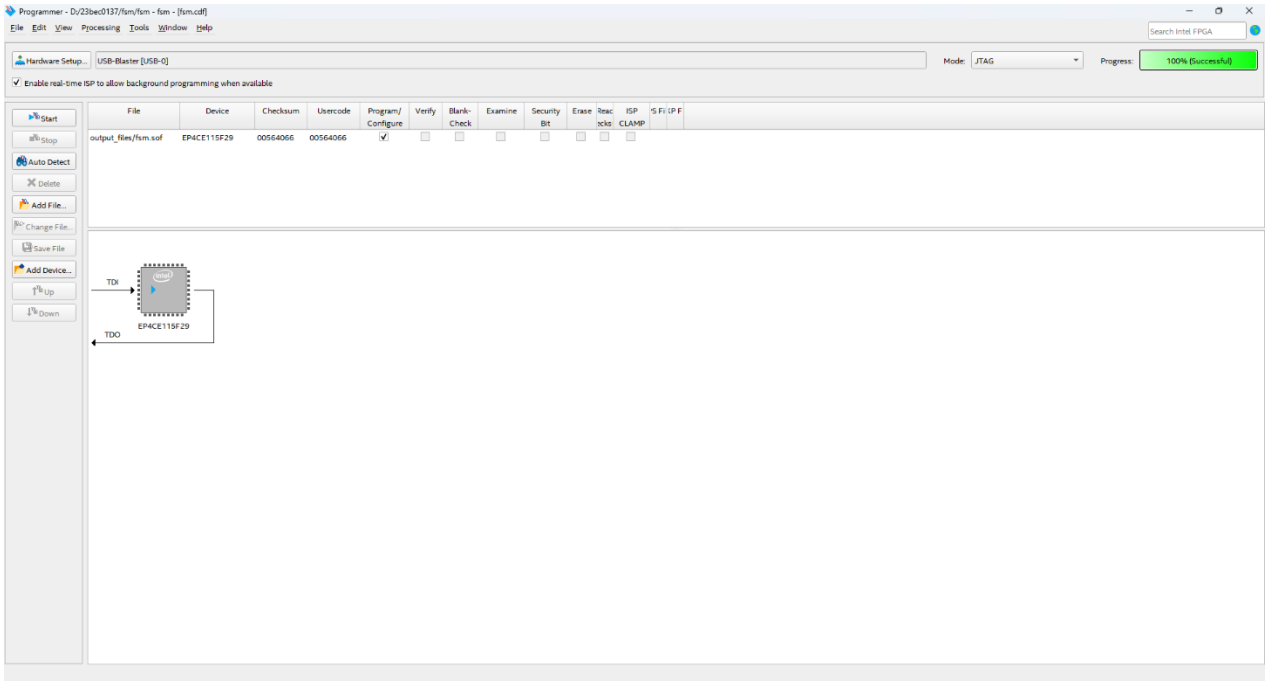
(5 for 3 clks, 10 for 2 clks)

RTL SCHEMATIC:

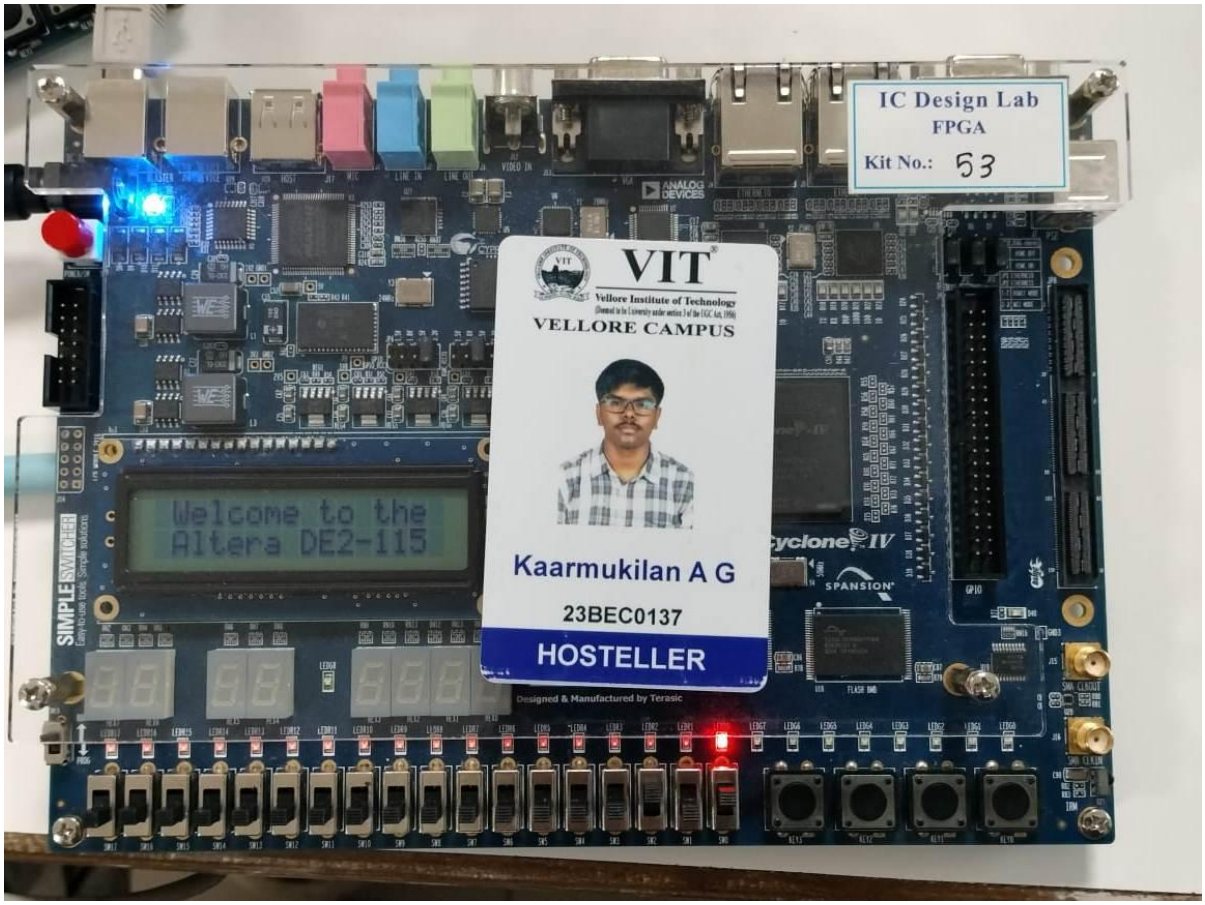


SYNTHESIS REPORT SUMMARY





FPGA IMPLEMENTATION:



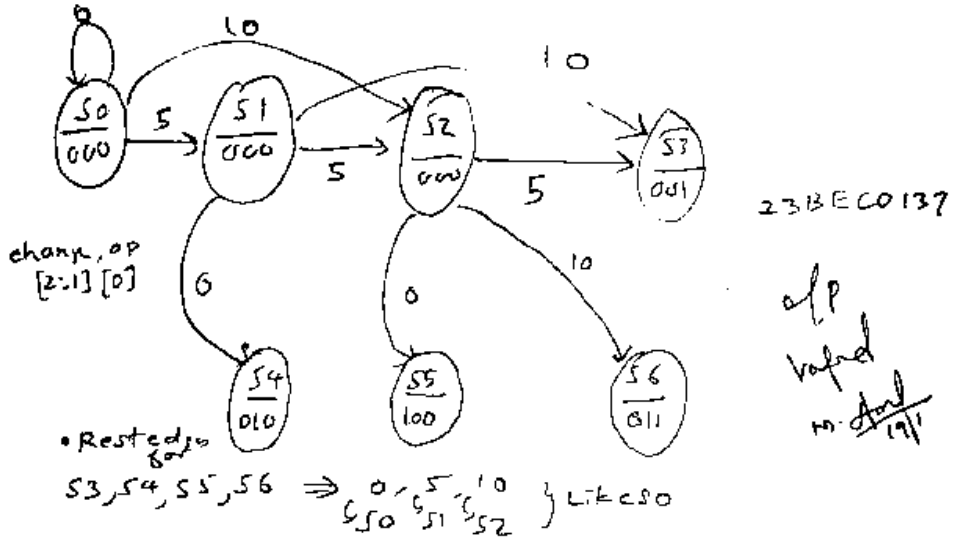
out=1(LEDR0),change=0(LEDG[1:0]- after 3 5rs(15rs) in 3 clk cycles

INFERENCE:

Hence, Vending machine -mealy is simulated and verified

VENDING MACHINE – MOORE

ATTESTATION - MOORE



RTL CODE AND TESTBENCH:

```
//23BEC0137 KAARMUKILAN A G
module moore_vending_machine(rst,clk,x,out,change);

input [1:0]x;
input rst,clk;
output out;
output [1:0]change;

reg [2:0] temp;

parameter S0 = 3'b000,
S1=3'b001,S2=3'b010,
S3=3'b011,S4=3'b100,
S5=3'b101,S6=3'b110;

reg [2:0]p_state,n_state;

always @(posedge clk or posedge rst) begin
    if(rst)
       p_state <= S0;
    else
       p_state <= n_state;
end

always @(*) begin
    n_state = p_state;
    case(p_state)

    S0,S3,S4,S5,S6: begin
        if(x==2'b00) begin
            n_state = S0;        end
        else if(x==2'b01) begin
            n_state = S1;        end
        else if(x==2'b10) begin
            n_state = S2;        end
        else begin
            n_state = S0;        end
    end

    S1: begin
        if(x==2'b00) begin
            n_state = S4;    end
        else if(x==2'b01) begin
            n_state = S2;    end
```

```
        if(x==2'b00) begin
            n_state = S4;    end
        else if(x==2'b01) begin
            n_state = S2;    end
        else if(x==2'b10) begin
            n_state = S3;   end
        else begin
            n_state = S0;   end
    end
    S2: begin
        if(x==2'b00) begin
            n_state = S5; end
        else if(x==2'b01) begin
            n_state = S3; end
        else if(x==2'b10) begin
            n_state = S6; end
        else begin
            n_state = S0;
        end
    end
    default n_state = S0;
    endcase

end

// o/p LOGIC
always @(*) begin
    case(p_state)
        S0: temp = 3'b000;
        S1: temp = 3'b000;
        S2: temp = 3'b000;
        S3: temp = 3'b001;
        S4: temp = 3'b010;
        S5: temp = 3'b100;
        S6: temp = 3'b011;
        default: temp = 3'b000;
    endcase
end

assign out = temp[0];
assign change = temp[2:1];

endmodule
```

```
module tb_moore_vending_machine;
reg clk;
reg rst;
reg [1:0] x;
wire out;
wire [1:0] change;

moore_vending_machine dut (rst,clk,x,out,change);

always #5 clk = ~clk;

    initial begin
        // Init
        clk = 0;
        rst = 1;
        x   = 2'b00;

        // Apply reset
        #12;
        rst = 0;
//Apply 5rs for 3 clks
        x = 2'b01;
        #30;   // 3 clock cycles

        // Apply 10 Rs (10) for 3 clocks

        x = 2'b10;
        #30;   // 3 clock cycles

        #20;
        $stop;
    end

endmodule
```

//23BEC0137 KAARMUKILAN A G

```
module moore_vending_machine(rst,clk,x,out,change);

input [1:0]x;
input rst,clk;
output out;
output [1:0]change;

reg [2:0] temp;

parameter S0 = 3'b000,
S1=3'b001,S2=3'b010,
S3=3'b011,S4=3'b100,
S5=3'b101,S6=3'b110;

reg [2:0]p_state,n_state;

always @(posedge clk or posedge rst) begin
   if(rst)
      p_state <= S0;
   else
      p_state <= n_state;
end

always @(*) begin
   n_state = p_state;
   case(p_state)
```

```
    S0,S3,S4,S5,S6: begin
        if(x==2'b00) begin
            n_state = S0;       end
        else if(x==2'b01) begin
            n_state = S1;       end
        else if(x==2'b10) begin
            n_state = S2;       end
        else begin
            n_state = S0;       end
    end

    S1: begin
        if(x==2'b00) begin
            n_state = S4;    end
        else if(x==2'b01) begin
            n_state = S2;    end
        else if(x==2'b10) begin
            n_state = S3;   end
        else begin
            n_state = S0;   end
    end
    S2: begin
        if(x==2'b00) begin
            n_state = S5; end
        else if(x==2'b01) begin
            n_state = S3; end
        else if(x==2'b10) begin
            n_state = S6; end
        else begin
            n_state = S0;
        end
    end
    default n_state = S0;
    endcase

end
```

```
// o/p LOGIC
always @(*) begin
    case(p_state)
        S0: temp = 3'b000;
        S1: temp = 3'b000;
        S2: temp = 3'b000;
        S3: temp = 3'b001;
        S4: temp = 3'b010;
        S5: temp = 3'b100;
        S6: temp = 3'b011;
        default: temp = 3'b000;
    endcase
end

assign out = temp[0];
assign change = temp[2:1];

endmodule


module tb_moore_vending_machine;
reg clk;
reg rst;
reg [1:0] x;
wire out;
wire [1:0] change;

moore_vending_machine dut (rst,clk,x,out,change);

always #5 clk = ~clk;

    initial begin
        // Init
        clk = 0;
        rst = 1;
```

```
        x   = 2'b00;

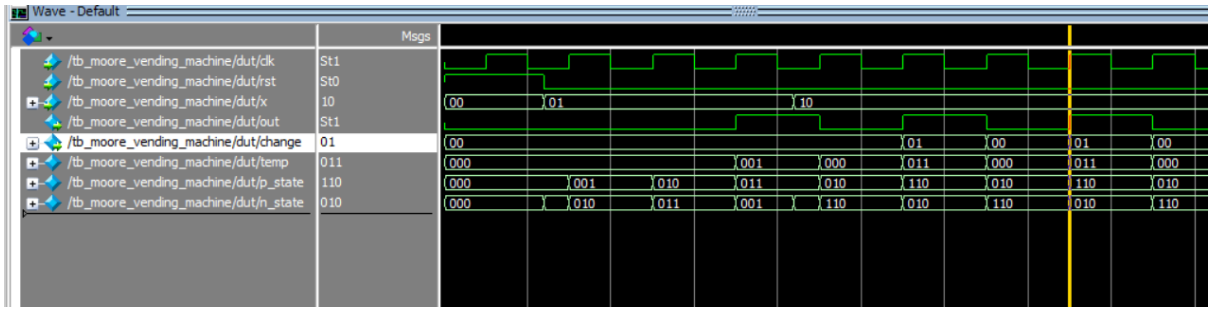
        // Apply reset
        #12;
        rst = 0;
//Apply 5rs for 3 clks
        x = 2'b01;
        #30;   // 3 clock cycles

        // Apply 10 Rs (10) for 3 clocks
        x = 2'b10;
        #30;   // 3 clock cycles

        #20;
        $stop;
    end

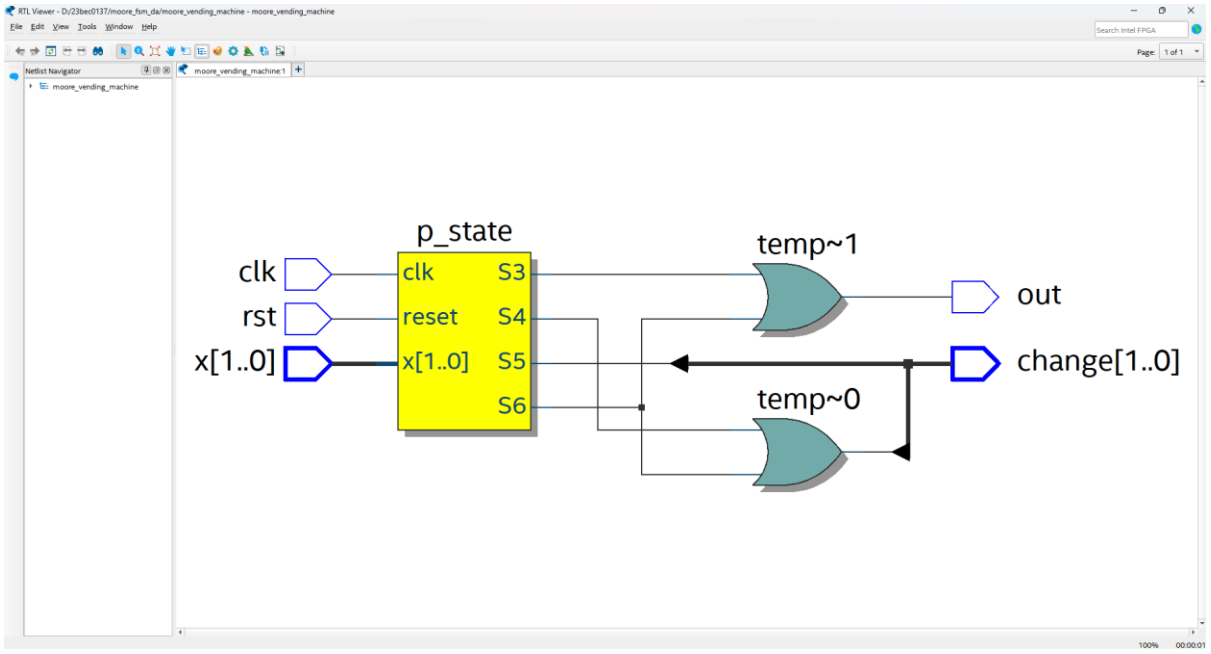
endmodule
```

FUNCTIONAL SIMULATION:

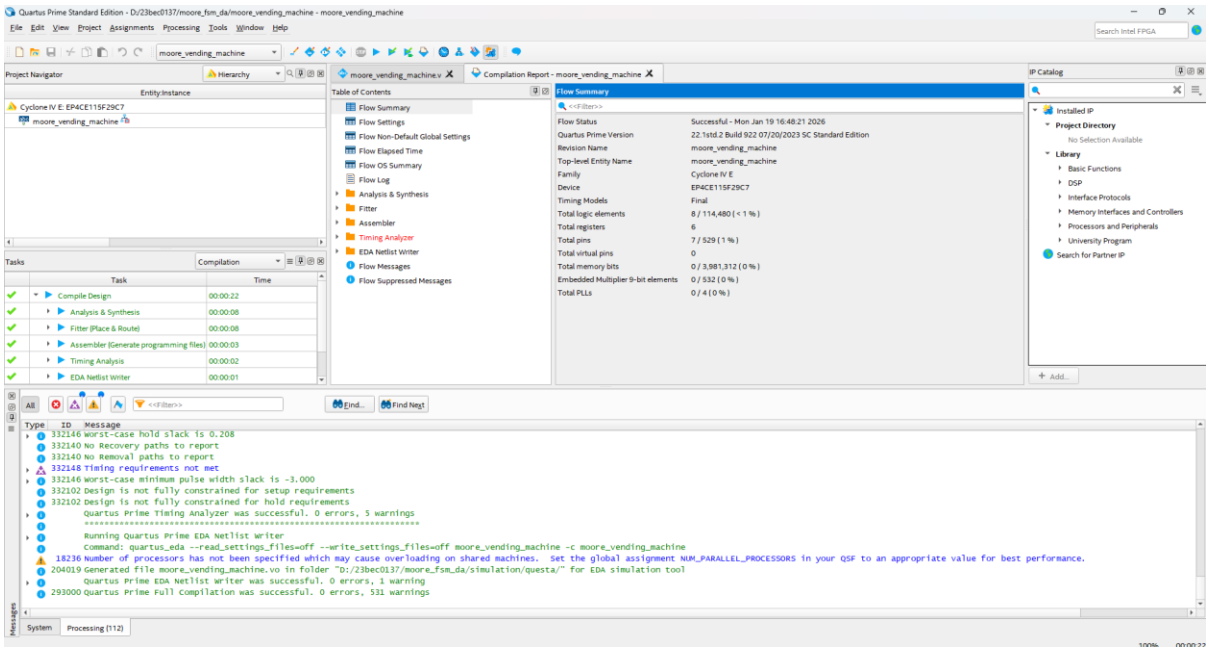


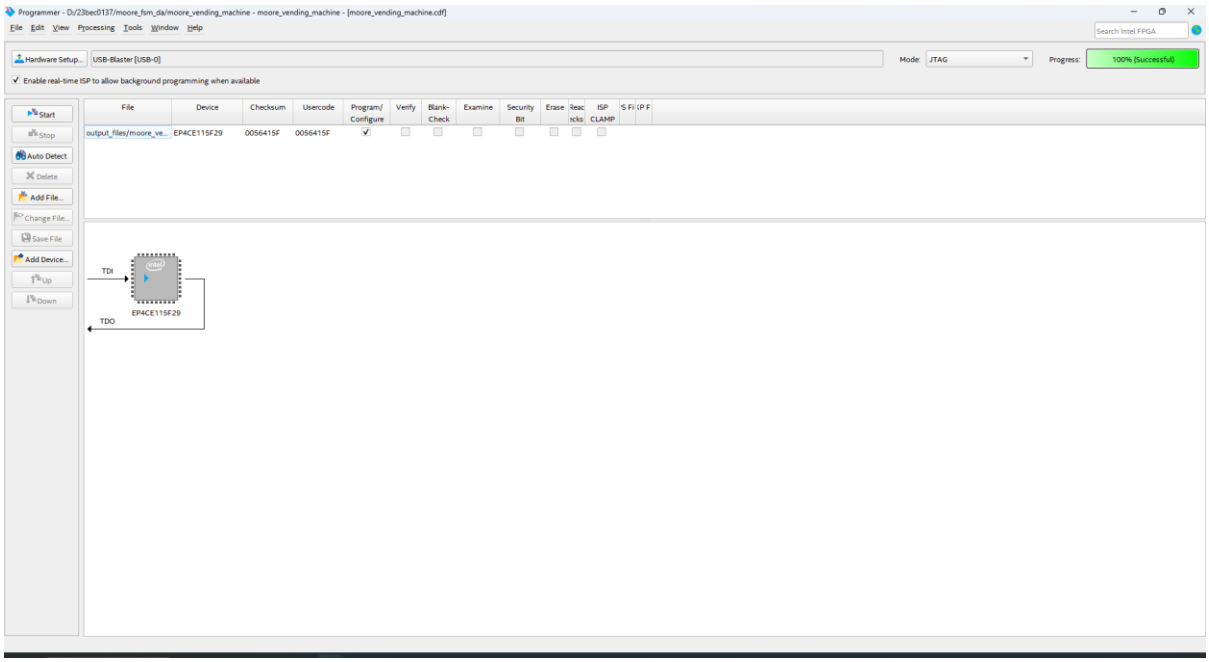
(5 for 3 clks, 10 for 2 clks)

RTL SCHEMATIC:

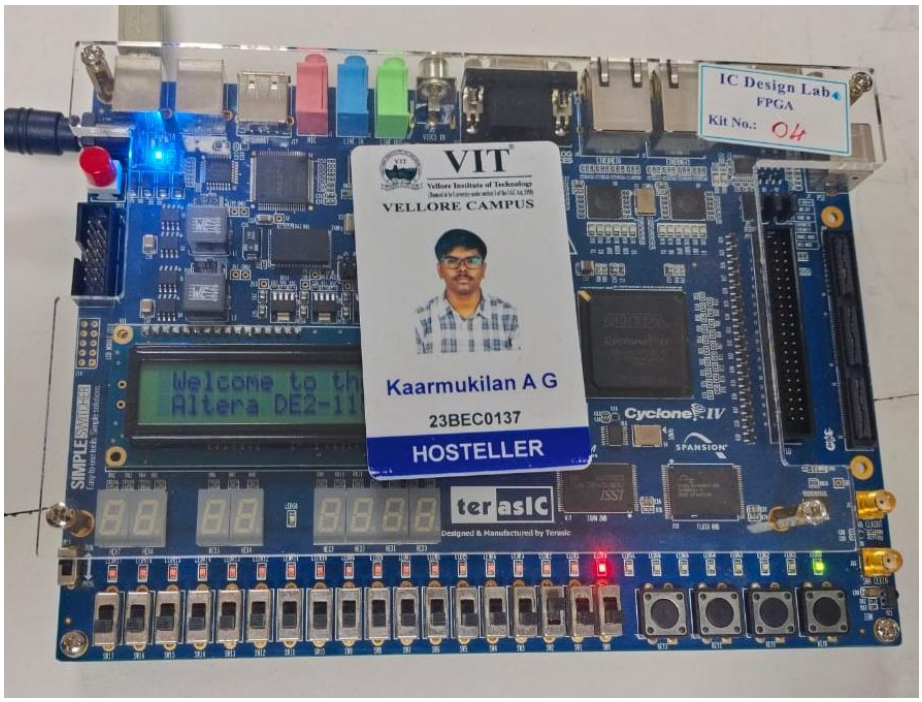


SYNTHESIS REPORT SUMMARY

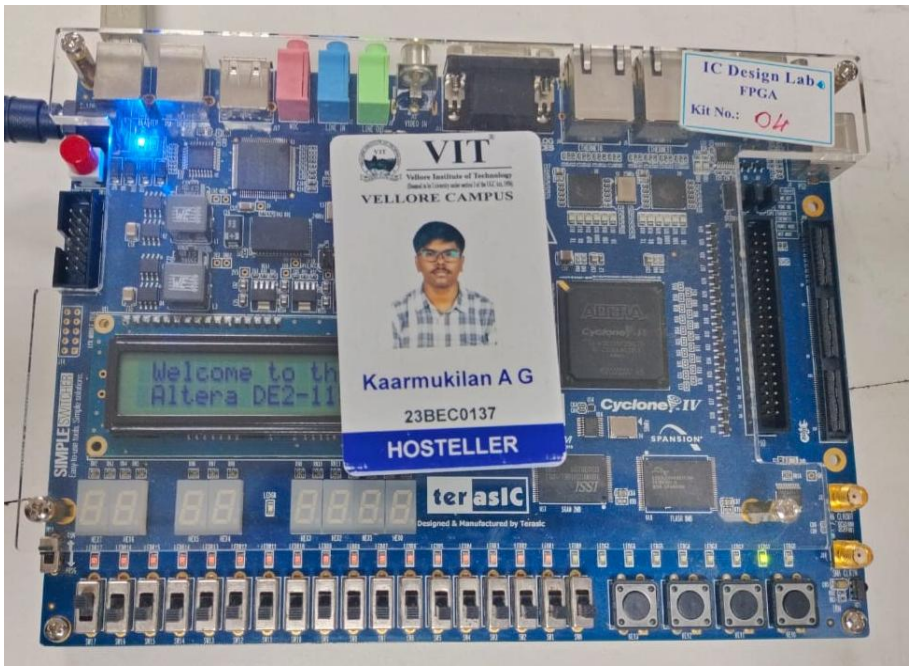




FPGA IMPLEMENTATION:



out=1(LEDR0),change=5(01-LEDG[1:0]- after 2 10rs(20rs(15+5 change) in 2 clk cycles



out=0(LEDR0),change=10(10-LEDG[1:0]- after 10rs,followed by 0rs (Change=10rs) in 2 clk cycles

INFERENCE:

Hence, Vending machine –Moore is simulated and verified